Lexical Analyzer Tool for Identifying Tokens, Keywords, Operators and Identifiers

REVIEW – 3

Batch 2 - Section 4

Team members

Roll No	Name
2420030427	Y. Nimisha
2420030498	K. Sanuhya
2420030505	A. Nikitha
2420030605	Y. Revanth

Under the guidance of

K S Niraja

Associate Professor

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Table of contents

- Abstract
- Problem Statement
- Existing System
- Proposed System
- Proposed Workflow
- Implementation
- Output

(DEEMED TO BE UNIVERSITY)

Abstract

- A lexical analyzer is the first phase of a compiler that processes the source program and converts the sequence of characters into meaningful tokens. This project aims to develop a Lexical Analyzer Tool that automatically identifies and classifies different components of a source program, such as keywords, identifiers, operators, constants, special symbols, and delimiters.
- The proposed tool accepts source code as input, scans it character by character, and generates a structured list of tokens along with their corresponding categories. It can also detect invalid or unrecognized symbols and provide appropriate error information. Regular expressions and lexical rules are used to define and recognize different token patterns.
- The system provides a simple and user-friendly interface that helps students and beginners understand how lexical analysis works internally in a compiler. It reduces the effort required for manual token identification and provides a practical visualization of the lexical analysis phase.

(DEEMED TO BE UNIVERSITY)

Problem Statement & Existing System

Problem Statement

- The problem is to develop a tool that takes source code as input and automatically identifies and categorizes its tokens using lexical analysis rules.

Existing System

- In the existing approach, lexical analysis is mainly performed as a part of a compiler or studied manually using examples and theoretical concepts.

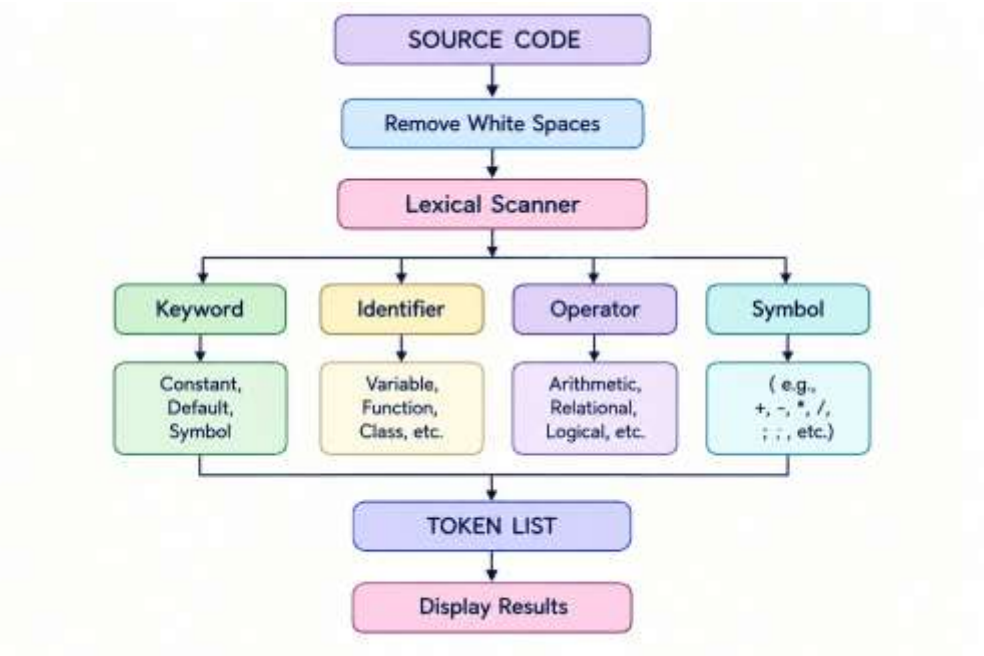
Existing system limitations:

- Token identification is often done manually while learning.
- There is no simple interactive visualization specifically for beginners.
- Understanding how characters are grouped into tokens can be difficult.
- Manual identification may result in classification errors.
- Invalid or unrecognized tokens may not be clearly highlighted.
- Students have limited opportunity to experiment with their own source code.
- General-purpose compiler tools hide the internal tokenization process from the user.

(DEEMED TO BE UNIVERSITY)

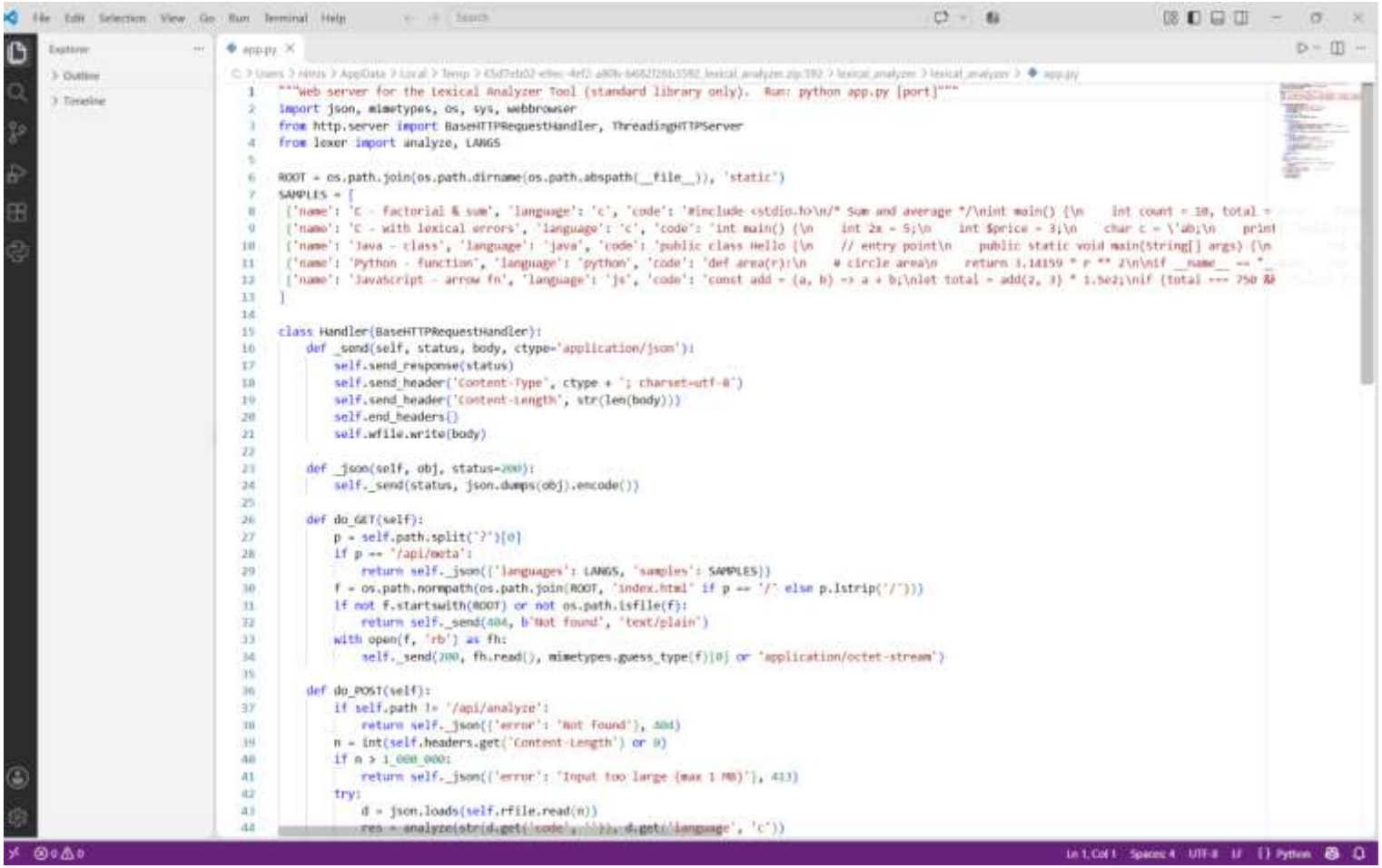
Proposed System & Workflow

- The proposed system is a **standalone Lexical Analyzer Tool** that allows users to enter or upload source code and automatically perform lexical analysis.
- The system scans the input and identifies different token categories using predefined lexical rules and regular expressions.



(DEEMED TO BE UNIVERSITY)

Implementation



```python
"""Web server for the Lexical Analyzer Tool (standard library only).  Run: python app.py [port]"""
import json, mimetypes, os, sys, webbrowser
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from lexer import analyze, LANGS

ROOT = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'static')
SAMPLES = [
 {'name': 'C - factorial & sum', 'language': 'c', 'code': '#include <stdio.h>\n/* Sum and average */\nint main() {\n    int count = 10, total =
 {'name': 'C - with lexical errors', 'language': 'c', 'code': 'int main() {\n    int 2x = 5;\n    int $price = 3;\n    char c = \'ab;\n    print
 {'name': 'Java - class', 'language': 'java', 'code': 'public class Hello {\n    // entry point\n    public static void main(String[] args) {\n
 {'name': 'Python - function', 'language': 'python', 'code': 'def area(r):\n    # circle area\n    return 3.14159 * r ** 2\n\nif __name__ == "_
 {'name': 'JavaScript - arrow fn', 'language': 'js', 'code': 'const add = (a, b) => a + b;\nlet total = add(2, 3) * 1.5e2;\nif (total === 750 &&
]

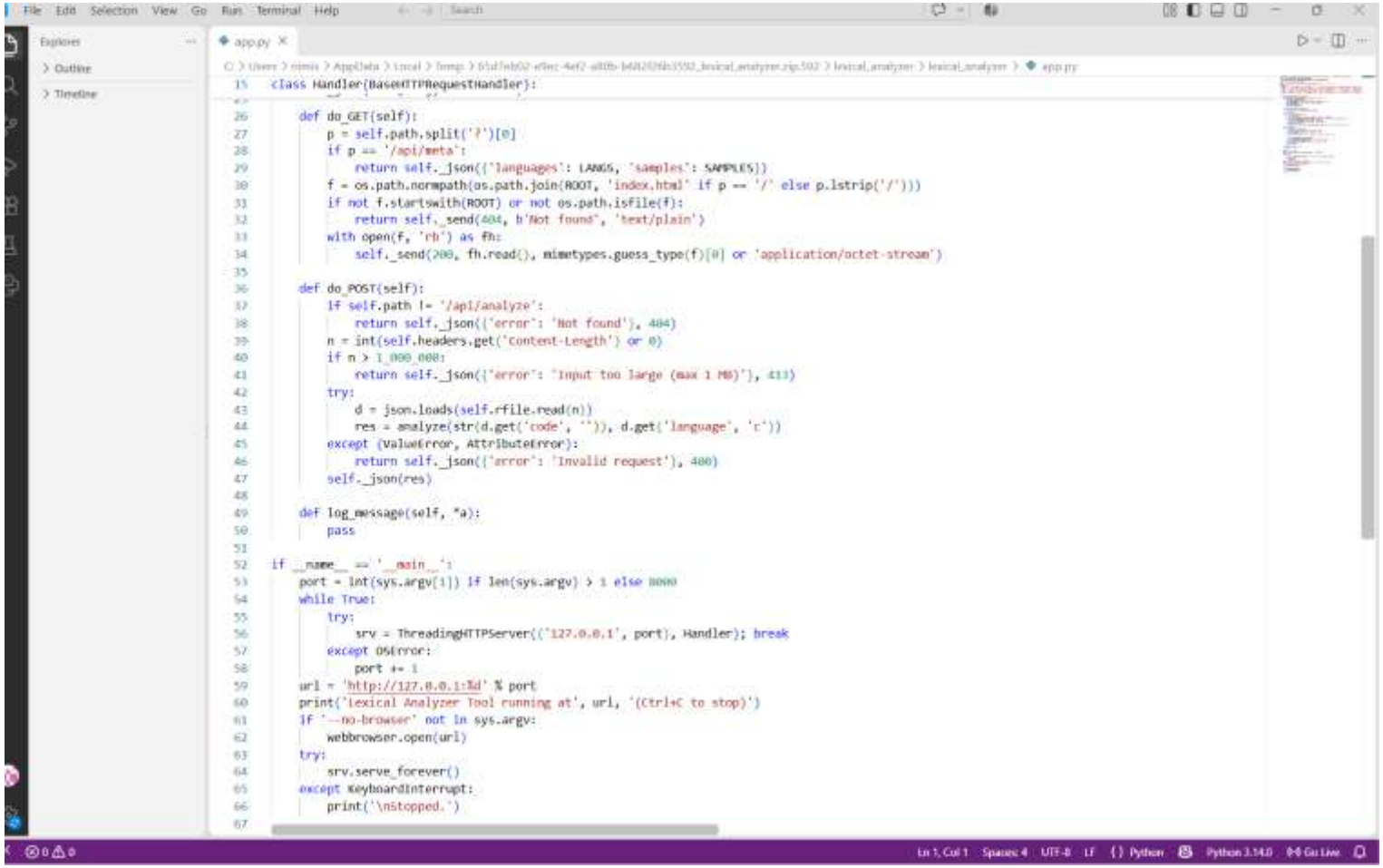
class Handler(BaseHTTPRequestHandler):
    def _send(self, status, body, ctype='application/json'):
        self.send_response(status)
        self.send_header('Content-Type', ctype + '; charset=utf-8')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def _json(self, obj, status=200):
        self._send(status, json.dumps(obj).encode())

    def do_GET(self):
        p = self.path.split('?')[0]
        if p == '/api/meta':
            return self._json({'languages': LANGS, 'samples': SAMPLES})
        f = os.path.normpath(os.path.join(ROOT, 'index.html' if p == '/' else p.lstrip('/')))
        if not f.startswith(ROOT) or not os.path.isfile(f):
            return self._send(404, b'Not found', 'text/plain')
        with open(f, 'rb') as fh:
            self._send(200, fh.read(), mimetypes.guess_type(f)[0] or 'application/octet-stream')

    def do_POST(self):
        if self.path != '/api/analyze':
            return self._json({'error': 'Not found'}, 404)
        n = int(self.headers.get('Content-Length') or 0)
        if n > 1_000_000:
            return self._json({'error': 'Input too large (max 1 MB)'}, 413)
        try:
            d = json.loads(self.rfile.read(n))
            res = analyze(str(d.get('code', '')), d.get('language', 'c'))
```

Implementation



```
class Handler(BaseHTTPRequestHandler):

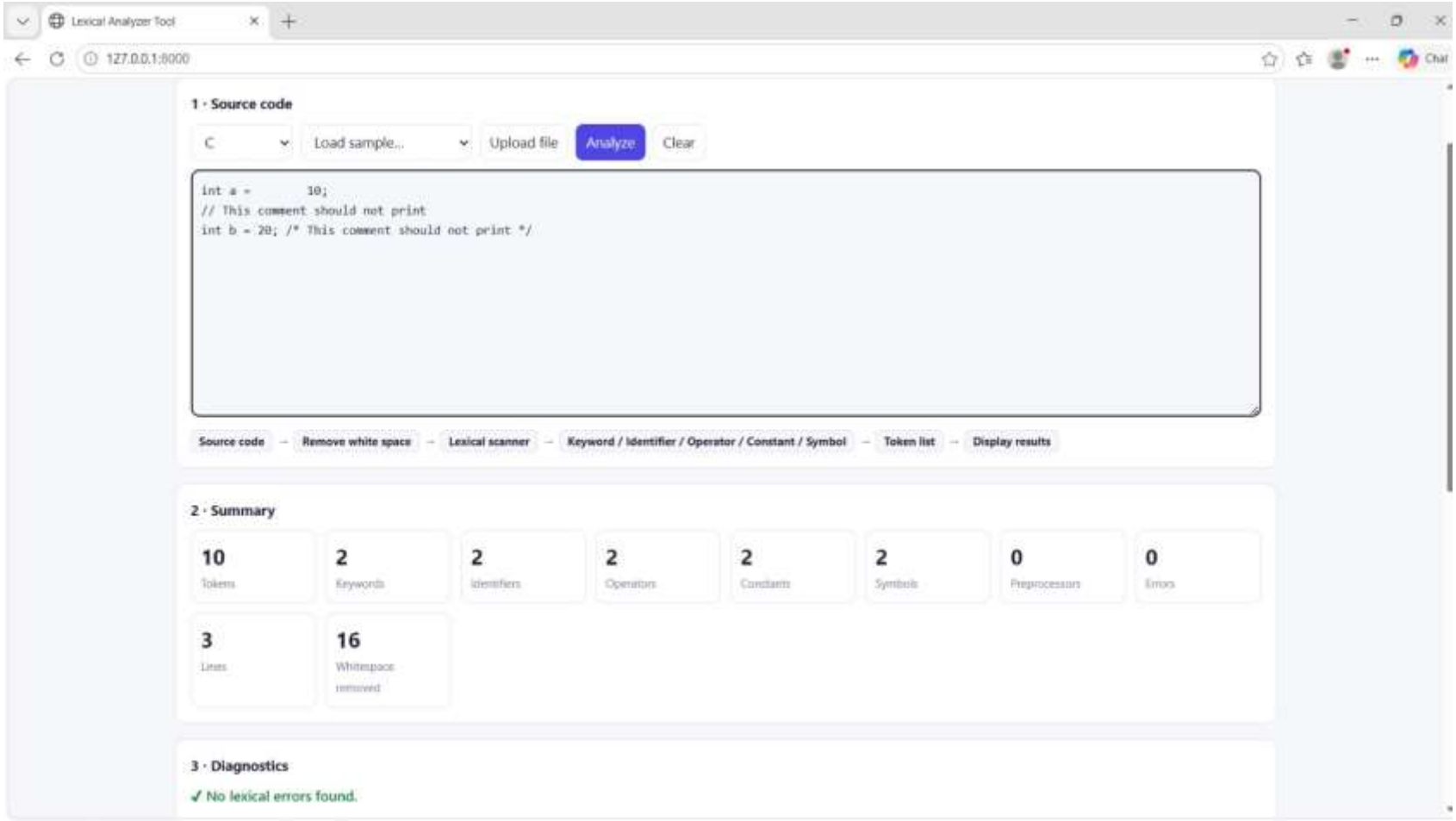
    def do_GET(self):
        p = self.path.split('?')[0]
        if p == '/api/meta':
            return self._json({'languages': LANGS, 'samples': SAMPLES})
        f = os.path.normpath(os.path.join(ROOT, 'index.html' if p == '/' else p.lstrip('/')))
        if not f.startswith(ROOT) or not os.path.isfile(f):
            return self._send(404, b'Not found', 'text/plain')
        with open(f, 'rb') as fh:
            self._send(200, fh.read(), mimetypes.guess_type(f)[0] or 'application/octet-stream')

    def do_POST(self):
        if self.path != '/api/analyze':
            return self._json({'error': 'Not found'}, 404)
        n = int(self.headers.get('Content-Length') or 0)
        if n > 1_000_000:
            return self._json({'error': 'Input too large (max 1 MB)'}, 413)
        try:
            d = json.loads(self.rfile.read(n))
            res = analyze(str(d.get('code', '')), d.get('language', 'c'))
        except (ValueError, AttributeError):
            return self._json({'error': 'Invalid request'}, 400)
        self._json(res)

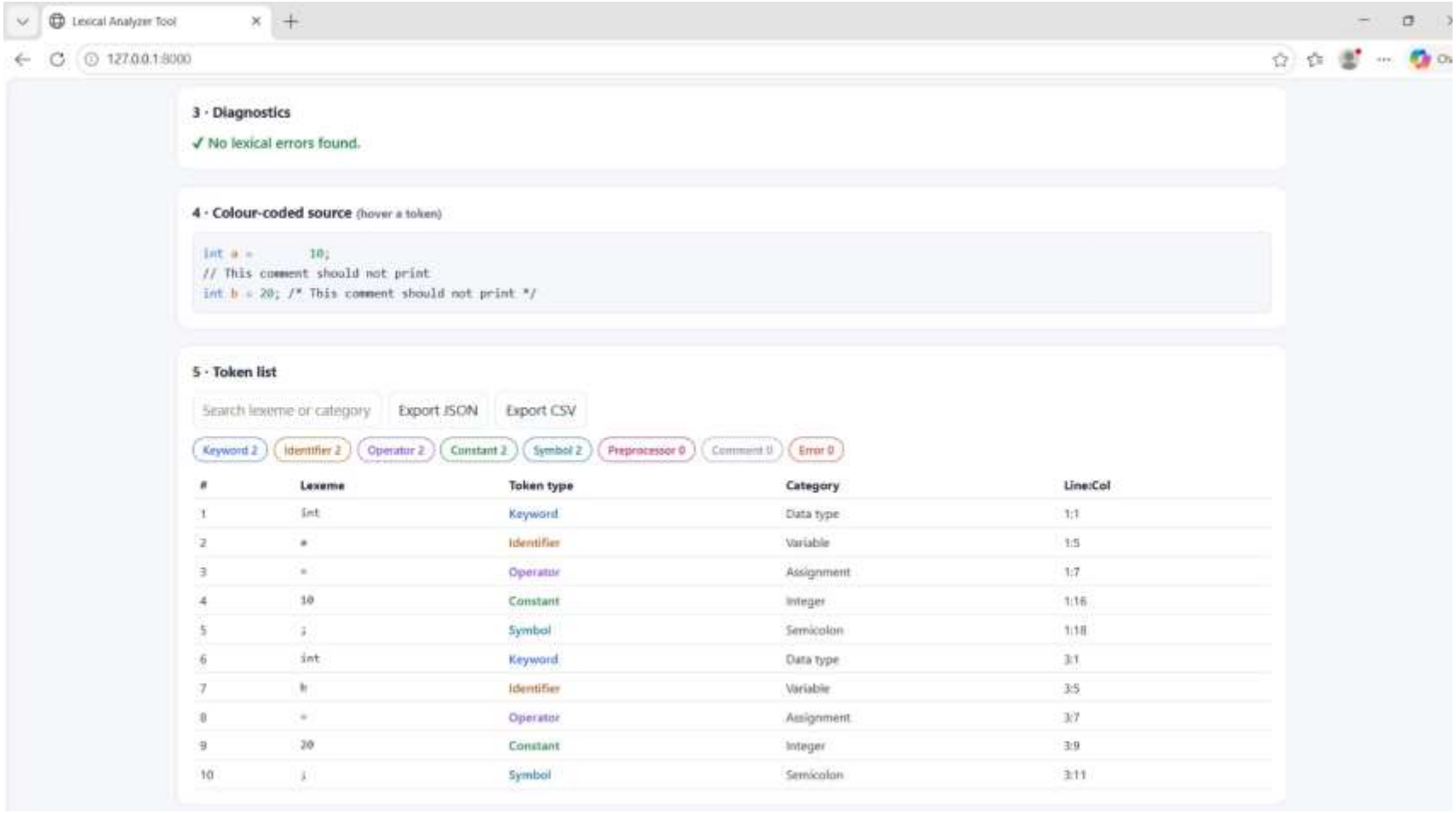
    def log_message(self, *a):
        pass

if __name__ == '__main__':
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 8000
    while True:
        try:
            srv = ThreadingHTTPServer(('127.0.0.1', port), Handler); break
        except OSError:
            port += 1
    url = 'http://127.0.0.1:%d' % port
    print('Lexical Analyzer Tool running at', url, '(Ctrl+C to stop)')
    if '--no-browser' not in sys.argv:
        webbrowser.open(url)
    try:
        srv.serve_forever()
    except KeyboardInterrupt:
        print('\nStopped.')
```

Output


Lexical Analyzer Tool
127.0.0.1:8000
1 · Source code
C
Load sample...
Upload file
Analyze
Clear
int a = 10;
// This comment should not print
int b = 20; /* This comment should not print */
Source code → Remove white space → Lexical scanner → Keyword / Identifier / Operator / Constant / Symbol → Token list → Display results
2 · Summary
10 Tokens
2 Keywords
2 Identifiers
2 Operators
2 Constants
2 Symbols
0 Preprocessors
0 Errors
3 Lines
16 Whitespace removed
3 · Diagnostics
✓ No lexical errors found.

Output



#	Lexeme	Token type	Category	Line:Col
1	int	Keyword	Data type	1:1
2	a	Identifier	Variable	1:5
3	=	Operator	Assignment	1:7
4	10	Constant	Integer	1:16
5	;	Symbol	Semicolon	1:18
6	int	Keyword	Data type	3:1
7	b	Identifier	Variable	3:5
8	=	Operator	Assignment	3:7
9	20	Constant	Integer	3:9
10	;	Symbol	Semicolon	3:11